SECTION A – DEBUGGING QUESTIONS

Question 1: Copy Constructor Pass-by-Value Infinite Recursion Trap

1. State the exact compiler error generated by Line 17.

The compiler reports an error similar to:

Invalid constructor; you cannot define a
copy constructor that takes its argument by value.

2. Why does passing source by value cause infinite recursive calls?

- A copy constructor creates a new object from an existing object.
- Passing the source object by value requires the compiler to create a copy of that object.
- To create that copy, the same copy constructor must be called again.
- This creates an infinite recursive copying process.
- Therefore, C++ requires a copy constructor to take its argument by reference.

3. Corrected copy constructor signature and importance of const and &

DynamicString(const DynamicString &source)

- & passes the object by reference and avoids creating another copy.
- const prevents accidental modification of the original object.
- It also allows copying from constant objects.

4. Complete Corrected C++ Code

```
#include <iostream>
#include <cstring>
using namespace std;

class DynamicString {
private:
   char *buffer;
   int length;

public:
```

```
    DynamicString(const char *str) {
        length = strlen(str);
        buffer = new char[length + 1];
        strcpy(buffer, str);
    }

    DynamicString(const DynamicString &source) {
        length = source.length;
        buffer = new char[length + 1];
        strcpy(buffer, source.buffer);
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        cout << buffer << endl;
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;

    s2.print();

    return 0;
}
```

Output:

Antigravity C++

Question 2: Shallow Copy & Double-Free Runtime Crash

1. Trace the memory address of samples before, during and after processAudio(track).

- Initially, track allocates an array in heap memory.
- The default copy constructor copies the pointer address instead of creating a new array.
- Both objects point to the same memory location.

- When the copied object is destroyed, the memory is released.
- The original object now contains a dangling pointer.

2. Why does the program crash with double-free?

- The copied object and original object share the same heap memory.
- When the copied object is destroyed, delete[] releases the memory.
- When the original object is destroyed, it attempts to release the same memory again.
- This causes a double-free error and heap corruption.

3. Corrected Deep Copy Constructor

4. Complete Corrected C++ Code

```
#include <iostream>
using namespace std;

class AudioBuffer {
public:
   int *samples;
   int capacity;

   AudioBuffer(int cap) {
      capacity = cap;
      samples = new int[capacity];

      for (int i = 0; i < capacity; i++) {
         samples[i] = i * 10;
      }

      cout << "Buffer allocated at: " << samples << endl;
   }

   AudioBuffer(const AudioBuffer &obj) {
      capacity = obj.capacity;
      samples = new int[capacity];

      for (int i = 0; i < capacity; i++) {
         samples[i] = obj.samples[i];
      }

      cout << "Copy allocated at: " << samples << endl;
   }
```

```
    ~AudioBuffer() {
        cout << "Deallocating buffer at: " << samples << endl;
        delete[] samples;
    }
};

void processAudio(AudioBuffer buf) {
    cout << "Processing sample 0: "
         << buf.samples[0] << endl;
}

int main() {
    AudioBuffer track(5);

    processAudio(track);

    cout << "Back in main, sample 0: "
         << track.samples[0] << endl;

    return 0;
}
```

5. Dry Run and Expected Output

Step	Operation	Result
1	Create track	Allocate 5 integers
2	Call processAudio	Create deep copy
3	Copy samples	0, 10, 20, 30, 40
4	Process sample 0	0
5	Destroy copied object	Release copied memory
6	Return to main	Original sample remains 0

7	Destroy track	Release original memory

Output (memory addresses will vary):

```
Buffer allocated at: 0x...
Copy allocated at: 0x...
Processing sample 0: 0
Deallocating buffer at: 0x...
Back in main, sample 0: 0
Deallocating buffer at: 0x...
```

Both memory addresses are different because deep copying creates separate memory.

Question 3: Stream Insertion Operator << Member Function Violation

1. Why does std::cout << f1 fail when operator << is a member of Fraction?

A binary member operator is called using:

left.operator@(right);

Therefore:

std::cout << f1;

is interpreted as:

std::cout.operator<<(f1);

But the overloaded member function belongs to Fraction, not std::ostream. Hence, the expression fails.

2. Why does f1 << std::cout compile?

It is interpreted as:

f1.operator<<(std::cout);

Since operator<< is a member of Fraction, this expression is valid.

However, it reverses the normal stream insertion syntax and is not suitable for standard output operations.

3. Corrected operator as a friend function

friend ostream& operator<<(ostream &os, const Fraction &f);

4. Corrected C++ Code

```
#include <iostream>
using namespace std;

class Fraction {
private:
   int numerator;
   int denominator;

public:
   Fraction(int n, int d) {
      numerator = n;
      denominator = d;
   }

   friend ostream& operator<<(ostream &os, const Fraction &f) {
      os << f.numerator << "/" << f.denominator;
      return os;
   }
};

int main() {
   Fraction f1(3, 4);

   cout << f1 << endl;

   return 0;
}
```

Output:

3/4

Question 4: Prefix vs. Postfix Increment Signature Inversion

1. Standard C++ signatures and return types

Operator	Signature	Return Type
Prefix	operator++()	Reference (StepCounter&)
Postfix	operator++(int)	Value (StepCounter)

2. Purpose of the dummy int parameter

- The dummy integer parameter distinguishes postfix increment from prefix increment.
- It is not used for calculations.
- Prefix has no argument, whereas postfix has an integer argument.

3. Why does prefix return by reference and postfix by value?

- Prefix increments the object first and returns the modified object by reference.
- Postfix saves the old value, increments the object and returns the old copy by value.
- This preserves the original postfix value.

4. Corrected C++ Code

```
#include <iostream>
using namespace std;

class StepCounter {
private:
   int steps;

public:
   StepCounter(int s = 0) {
      steps = s;
   }

   StepCounter& operator++() {
      steps++;
      return *this;
   }

   StepCounter operator++(int) {
      StepCounter temp = *this;
      steps++;
      return temp;
```

```
    }

    void display() const {
        cout << "Steps: " << steps << endl;
    }
};

int main() {
    StepCounter c1(10);

    StepCounter c2 = ++c1;
    c2.display();
    c1.display();

    StepCounter c3 = c1++;
    c3.display();
    c1.display();

    return 0;
}
```

Dry Run:

Operation	c1	c2	c3
Initial	10	-	-
c2 = ++c1	11	11	-
c3 = c1++	12	11	11

Output:

```
Steps: 11
Steps: 11
Steps: 11
Steps: 12
```

Question 5: Violation of Fundamental Operator Overloading Rules

1. Operators that cannot be overloaded

The following operators cannot be overloaded in C++:

Operator	Meaning
.	Member access
.*	Pointer-to-member access
::	Scope resolution
?:	Conditional operator
sizeof	Size operator

The dot operator cannot be overloaded because it is used for direct member access and is part of the language's built-in syntax.

2. Why is int operator+(int, int) rejected?

- C++ does not allow overloading operators for only built-in data types.
- At least one operand must be a user-defined type or enum type.
- Both operands here are integers, so the compiler rejects it.

3. Can programmers define new operators such as **?

No.

- C++ allows overloading only existing operators.
- New operator symbols such as ** cannot be created.
- Operator precedence, associativity and number of operands cannot be changed.

4. Corrected C++ Code

#include <iostream>

```
using namespace std;

class MathVector {
public:
    double x, y;

    MathVector(double a, double b) {
        x = a;
        y = b;
    }

    bool operator==(const MathVector &other) {
        return (x == other.x && y == other.y);
    }

    MathVector operator+(const MathVector &other) {
        return MathVector(x + other.x, y + other.y);
    }

    MathVector power(int p) {
        double a = 1, b = 1;

        for (int i = 0; i < p; i++) {
            a *= x;
            b *= y;
        }

        return MathVector(a, b);
    }
};

int main() {
    MathVector v1(2, 3), v2(2, 3);

    if (v1 == v2)
        cout << "Vectors are equal" << endl;

    MathVector v3 = v1 + v2;

    cout << "Addition: " << v3.x << ", " << v3.y << endl;

    MathVector v4 = v1.power(2);
```

```
    cout << "Power: " << v4.x << ", " << v4.y << endl;

    return 0;
}
```

Output:

Vectors are equal
Addition: 4, 6
Power: 4, 9

Note: The power() method raises each coordinate independently; it is not a vector exponentiation operator.

SECTION B – PSEUDO-CODE WRITING QUESTIONS

Question 1: Dynamic String Engine with Deep Copy & Overloaded +

1. Formal Structured Pseudocode

```
CLASS DynStr

PRIVATE:
    dataPtr : POINTER TO CHARACTER
    charCount : INTEGER

PUBLIC:

    CONSTRUCTOR(sourceString)

        charCount = LENGTH(sourceString)

        ALLOCATE ARRAY[charCount + 1] FOR dataPtr

        COPY sourceString INTO dataPtr

    END CONSTRUCTOR
```

```
COPY CONSTRUCTOR(REF other AS DynStr)

    charCount = other.charCount

    ALLOCATE ARRAY[charCount + 1] FOR dataPtr

    FOR i = 0 TO charCount
        dataPtr[i] = other.dataPtr[i]
    END FOR

END COPY CONSTRUCTOR


DESTRUCTOR()

    DEALLOCATE dataPtr
    dataPtr = NULL

END DESTRUCTOR


OPERATOR +(REF rhs AS DynStr) RETURNS DynStr

    totalLength = charCount + rhs.charCount

    CREATE temporary buffer[totalLength + 1]

    COPY dataPtr INTO temporary buffer

    APPEND rhs.dataPtr TO temporary buffer

    CREATE result AS DynStr(temporary buffer)

    RETURN result

END OPERATOR


METHOD Display()

    DISPLAY dataPtr
```

```
    END METHOD

END CLASS


MAIN

    CREATE strA = DynStr("Hello ")
    CREATE strB = DynStr("World!")

    strC = strA + strB

    DISPLAY "Concatenated String: "
    CALL strC.Display()

END MAIN
```

2. Dry-Run Trace Table

Given:

- strA = "Hello "
- strB = "World!"

Step	Operation	Result
1	Create strA	Allocate 7 characters
2	Create strB	Allocate 7 characters
3	Calculate total length	6 + 6 = 12
4	Allocate result buffer	13 characters
5	Copy strA	Hello
6	Append strB	World!
7	Final string	Hello World!

Expected Output

Concatenated String: Hello World!

Question 2: Complex Number System with Constructor Overloading & Operators

1. Structured Pseudocode

```
CLASS Complex

PRIVATE:
   real : REAL
   imag : REAL

PUBLIC:

   CONSTRUCTOR()
      real = 0.0
      imag = 0.0
   END CONSTRUCTOR

   CONSTRUCTOR(r)
      real = r
      imag = 0.0
   END CONSTRUCTOR

   CONSTRUCTOR(r, i)
      real = r
      imag = i
   END CONSTRUCTOR


   OPERATOR +(REF other AS Complex) RETURNS Complex

      RETURN Complex(real + other.real,
                     imag + other.imag)

   END OPERATOR


   OPERATOR *(REF other AS Complex) RETURNS Complex
```

```
        r = (real * other.real) - (imag * other.imag)

        i = (real * other.imag) + (imag * other.real)

        RETURN Complex(r, i)

    END OPERATOR


    OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN

        IF real = other.real AND imag = other.imag THEN
            RETURN TRUE
        ELSE
            RETURN FALSE
        END IF

    END OPERATOR


    METHOD Display()

        DISPLAY real, " + ", imag, "i"

    END METHOD

END CLASS


MAIN

    CREATE C1 = Complex(3, 2)
    CREATE C2 = Complex(1, 4)

    result = C1 * C2

    DISPLAY "Product: "
    CALL result.Display()

END MAIN
```

2. Step-by-Step Trace

Given:

$$ C_1 = 3+2i $$
$$ C_2 = 1+4i $$

Formula:

$$ (a+bi)(c+di)=(ac-bd)+(ad+bc)i $$

Calculation:

Step	Calculation	Result
1	3 × 1	3
2	2 × 4	8
3	3 − 8	-5
4	3 × 4	12
5	2 × 1	2
6	12 + 2	14

Final answer:

$$ \boxed{-5+14i} $$

Output:

Product: -5 + 14i

Question 3: Financial Currency Class with Stream I/O & Unary Negation

1. Structured Pseudocode

CLASS Currency

PRIVATE:
    dollars : INTEGER

```
    cents : INTEGER

PUBLIC:

    CONSTRUCTOR(d = 0, c = 0)

        total = (d * 100) + c

        dollars = total DIV 100
        cents = total MOD 100

    END CONSTRUCTOR


    OPERATOR -() RETURNS Currency

        RETURN Currency(-dollars, -cents)

    END OPERATOR


    FRIEND OPERATOR <<(REF outStream, REF cur AS Currency)

        DISPLAY "$", cur.dollars, ".", TWO_DIGIT(cur.cents)

        RETURN outStream

    END OPERATOR


    FRIEND OPERATOR >>(REF inStream, REF cur AS Currency)

        INPUT cur.dollars, cur.cents

        total = (cur.dollars * 100) + cur.cents

        cur.dollars = total DIV 100
        cur.cents = total MOD 100

        RETURN inStream

    END OPERATOR
```

```
END CLASS


MAIN
    CREATE cur AS Currency
    DISPLAY "Enter dollars and cents: "
    INPUT cur
    DISPLAY "Original Amount: ", cur
    negative = -cur
    DISPLAY "Negative Amount: ", negative
END MAIN
```

2. Dry-Run Trace Table

Given:

- Dollars = 5
- Cents = 275

Normalization:

$$ 275 \text{ cents} = 2 \text{ dollars } + 75 \text{ cents} $$
$$ 5+2=7 \text{ dollars} $$

Step	Operation	Dollars	Cents
1	Initial input	5	275
2	Normalize cents	7	75
3	Apply unary negation	-7	-75

Output:

Enter dollars and cents: 5 275
Original Amount: $7.75
Negative Amount: $-7.75

Question 4: Polynomial Expression with Overloaded Subscript [] & Destructor

1. Formal Structured Pseudocode

```
CLASS Polynomial

PRIVATE:
   degree : INTEGER
   coeffArray : POINTER TO REAL

PUBLIC:

   CONSTRUCTOR(deg)

      degree = deg

      ALLOCATE ARRAY[degree + 1] FOR coeffArray

      FOR i = 0 TO degree
         coeffArray[i] = 0.0
      END FOR

   END CONSTRUCTOR


   COPY CONSTRUCTOR(REF other AS Polynomial)

      degree = other.degree

      ALLOCATE ARRAY[degree + 1] FOR coeffArray

      FOR i = 0 TO degree
         coeffArray[i] = other.coeffArray[i]
      END FOR

   END COPY CONSTRUCTOR
```

```
DESTRUCTOR()

   DEALLOCATE coeffArray
   coeffArray = NULL

END DESTRUCTOR


OPERATOR [](index AS INTEGER) RETURNS REF REAL

   IF index >= 0 AND index <= degree THEN
      RETURN REF coeffArray[index]
   ELSE
      DISPLAY "Invalid Index"
      RETURN REF errorSink
   END IF

END OPERATOR


OPERATOR +(REF other AS Polynomial) RETURNS Polynomial

   maxDegree = MAX(degree, other.degree)

   CREATE result AS Polynomial(maxDegree)

   FOR i = 0 TO maxDegree

      value1 = 0
      value2 = 0

      IF i <= degree THEN
         value1 = coeffArray[i]
      END IF

      IF i <= other.degree THEN
         value2 = other.coeffArray[i]
      END IF

      result.coeffArray[i] = value1 + value2

   END FOR
```

```
        RETURN result

    END OPERATOR

END CLASS


MAIN

    CREATE P1 AS Polynomial(2)
    CREATE P2 AS Polynomial(3)

    P1[0] = 5
    P1[1] = 2
    P1[2] = 3

    P2[0] = 7
    P2[1] = 0
    P2[2] = 1
    P2[3] = 4

    P3 = P1 + P2

    DISPLAY "Result Polynomial"

    FOR i = 0 TO 3
        DISPLAY P3[i]
    END FOR

END MAIN
```

2. Dry-Run Trace Table

Given:

$$P_1(x)=3x^2+2x+5$$
$$P_2(x)=4x^3+x^2+7$$

Arrange coefficients in ascending powers:

Power	P1	P2	Addition

\(x^0\)	5	7	12
\(x^1\)	2	0	2
\(x^2\)	3	1	4
\(x^3\)	0	4	4

Final polynomial:

\[\boxed{P_3(x)=4x^3+4x^2+2x+12} \]

Output:

Result Polynomial:
4x^3 + 4x^2 + 2x + 12

Question 5: 2D Vector with Prefix/Postfix Increment & Compound Assignment

1. Structured Pseudocode

```
CLASS Vector2D

PRIVATE:
   x : REAL
   y : REAL

PUBLIC:

   CONSTRUCTOR(a = 0.0, b = 0.0)

      x = a
      y = b

   END CONSTRUCTOR


   OPERATOR ++() RETURNS REF Vector2D

      x = x + 1.0
```

```
        y = y + 1.0

        RETURN REF THIS

    END OPERATOR


    OPERATOR ++(dummy INTEGER) RETURNS Vector2D

        CREATE temp AS COPY OF THIS

        x = x + 1.0
        y = y + 1.0

        RETURN temp

    END OPERATOR


    OPERATOR +=(REF other AS Vector2D)
        RETURNS REF Vector2D

        x = x + other.x
        y = y + other.y

        RETURN REF THIS

    END OPERATOR


    METHOD Display()

        DISPLAY "(", x, ", ", y, ")"

    END METHOD

END CLASS


MAIN

    CREATE v1 = Vector2D(5.0, 8.0)
```

```
    v2 = ++v1

    DISPLAY "After Prefix Increment:"
    CALL v1.Display()
    CALL v2.Display()

    v3 = v1++

    DISPLAY "After Postfix Increment:"
    CALL v1.Display()
    CALL v3.Display()

    v1 += v2

    DISPLAY "After Compound Assignment:"
    CALL v1.Display()

END MAIN
```

2. Dry-Run Trace Table

Initial value:

$$ v_1=(5.0,8.0) $$

Operation 1: v2 = ++v1

Prefix increment increases both coordinates before copying the result.

$$ v_1=(6.0,9.0) $$
$$ v_2=(6.0,9.0) $$

Operation 2: v3 = v1++

Postfix increment copies the old value first, then increments the original.

$$ v_3=(6.0,9.0) $$
$$ v_1=(7.0,10.0) $$

Operation 3: v1 += v2

$$ v_1=(7+6,10+9) $$
$$ v_1=(13.0,19.0) $$

Final Dry-Run Table

Operation	v1	v2	v3
Initial	(5, 8)	–	–
v2 = ++v1	(6, 9)	(6, 9)	–
v3 = v1++	(7, 10)	(6, 9)	(6, 9)
v1 += v2	(13, 19)	(6, 9)	(6, 9)

Final Output:

After Prefix Increment:
(6, 9)
(6, 9)

After Postfix Increment:
(7, 10)
(6, 9)

After Compound Assignment:
(13, 19)